

+ Options					fruit_id	fruit_name
☐					1	mangoes
☐					2	oranges
☐					3	grapes
☐					4	pineapples
☐ Check all With selected: Edit Copy						

Figure 10: Storing sample data in the table

```
<?php
function connect()
{
    $host = 'localhost';
    $user = 'root';
    $pass = '';
    $db = 'test';

    $conn = new mysqli($host, $user, $pass, $db) or die("Connect failed: %s\n" . $conn -> error);

    return $conn;
}

function disconnect()
{
    $conn -> close();
}

?>
```

Figure 11: Code for *conn.php*

Using MySQL with Xampp

Now that we have launched a simple Web app using PHP, let's move on to a slightly more advanced concept, which is connecting your app to a database. If you remember, we had started two services in the Xampp control panel. One was the Apache Web server and the other was the MySQL database. MySQL is a free and open source relational database that is popularly used in many Web applications across the Internet.

We will use the PHPMyAdmin tool to access MySQL databases. This tool is written in PHP to handle the administration of MySQL. In order to access PHPMyAdmin, open a new tab in your Web browser and type *localhost/phpmyadmin* or *127.0.0.1/phpmyadmin*. This should open up the PHPMyAdmin interface.

On the left hand pane of PHPMyAdmin, you will be able to see a list of preloaded databases that you can use. Needless to say, you can create your own database, but for the purpose of this tutorial, let us use the preinstalled test database, as shown in Figure 8.

In order to demonstrate the usage of MySQL, we will create a simple table named 'fruits'. In this table, we will create two columns, namely *fruit_id*, which acts as a primary key identifier for each row and *fruit_name*. We will then try and connect the Web app with this database and retrieve the data from the fruits table. Figure 9 shows the table with two attributes, as discussed above.

After creating the table, let's fill it up with some sample data, as shown in Figure 10.

```
<?php
1  require('conn.php');
2
3
4  $conn = connect();
5
6  $sql = "SELECT * FROM 'fruits'";
7
8  $query = $conn -> query($sql);
9  while($result = $query -> fetch_array(MYSQLI_ASSOC))
10 {
11     echo $result['fruit_id'] . ' ' . $result['fruit_name'] . '<br>';
12 }
13
14 disconnect($conn);
15 ?>
```

Figure 12: Code for *index.php*

```
localhost
1 mangoes
2 oranges
3 grapes
4 pineapples
```

Figure 13: Data retrieved from the database

Connecting to the database

Back in your *htdocs* folder, let's create a new file and name it *conn.php*. This file will contain two functions, one for connecting to the database and the other for disconnecting. In *conn.php*, write the code, as shown Figure 11.

Now let's get back to the *index.php* file. The first thing we do is include the *conn.php* file into the *index.php* file using the *require()* function. We will then establish a connection with the database. Write the code shown in Figure 12 on your *index.php* file to establish a connection and retrieve the data from the database.

Now when you refresh the index page on your browser, you should be able to see the list of fruits retrieved from the database, as shown in Figure 13. Congratulations, you have just learned how to set up a MySQL database and retrieve data from it.

It is always a good practice to deploy and test your Web applications locally before launching them for public access. For the Windows platform, the Xampp package provides the easiest way to deploy and test PHP applications on your device. We encourage you to explore more ways in which you can utilise Xampp for your PHP projects. **END** 🐧

By: Mir H.S. Quadri

The author is a scholar and researcher in the fields of artificial intelligence and machine learning. He has worked on multiple projects using a wide array of frameworks. He is also a FOSS enthusiast and actively contributes to several open source projects. You can find his works on codelatte.site.